Cambridge (CIE) A Level Computer Science

Your notes

Artificial Intelligence (AI)

Contents

- AI & Graphs
- Dijkstra's Shortest Path Algorithm
- A* Algorithm
- Machine Learning

© 2025 Save My Exams, Ltd.

Your notes

Graphs to aid AI

What is a graph?

- A graph is a set of **vertices/nodes** connected by **edges/arcs**
- In AI, graphs are used to **model relationships, networks, and decision-making problems**
- They can represent things like:
  - Routes between cities (pathfinding)
  - Connections in a neural network
  - State transitions in decision trees or planning

Types of graphs

Type	Use in AI
Directed Graph	Models one-way relationships, such as **state transitions** in search algorithms (e.g. A*)
Undirected Graph	Useful for **bidirectional relationships**, such as undirected social connections in a graph
Weighted Graph	Represents **costs or heuristics**, commonly used in **shortest path algorithms** like Dijkstra's or A*

Representation of graphs

- Graphs can be represented using:
  - **Adjacency Matrix** – ideal for dense graphs and fast lookup
  - **Adjacency List** – more space, efficient for sparse graphs (often used in AI search)
- Both can be **used in AI to represent problem spaces** such as:
  - Game boards
  - Map-based navigation
  - Resource allocation networks

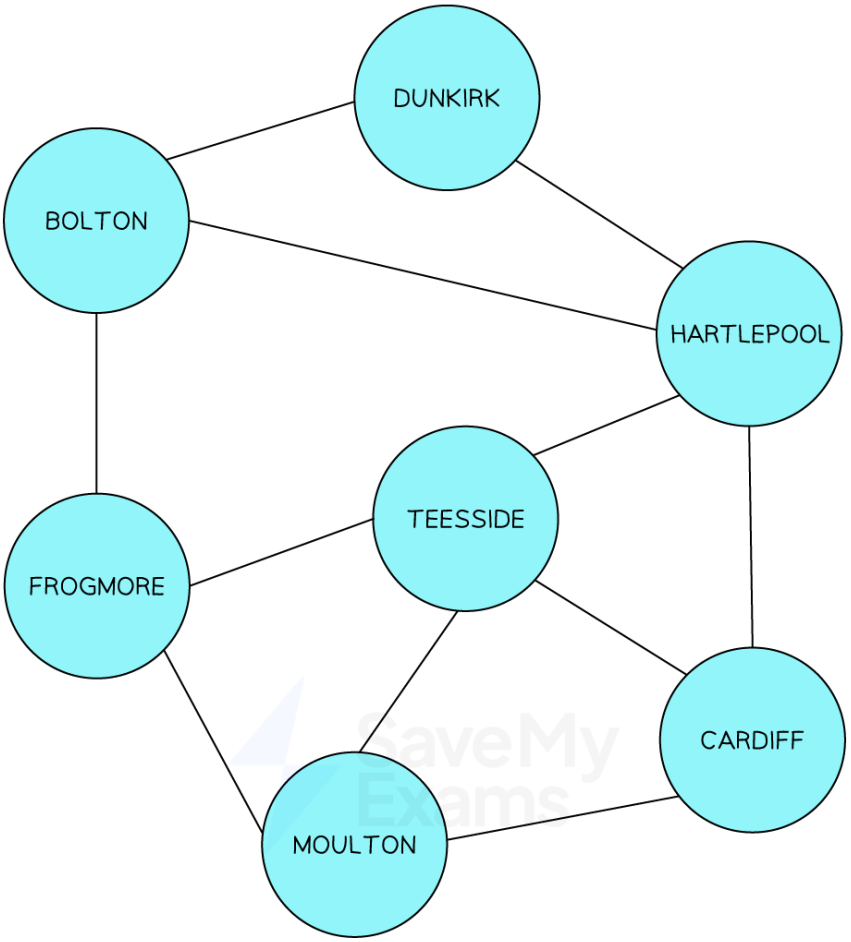
Undirected, unweighted graph

- An **undirected, unweighted graph** is a type of graph where:
  - **Edges have no direction** , connections go both ways

© 2025 Save My Exams, Ltd.

- **All edges are equal**, there are no weights or costs
- It is represented with a **symmetric adjacency matrix**, where a 1 indicates a connection and a 0 means no connection

Your notes



	B	C	D	F	H	M	T
B	0	0	1	1	1	0	0
C	0	0	0	0	1	1	1
D	1	0	0	0	1	0	0
F	1	0	0	0	0	1	1
H	1	1	1	0	0	0	1
M	0	1	0	1	0	0	1
T	0	1	0	1	1	1	0

Copyright © Save My Exams. All Rights Reserved

- In this example:
  - Each **node** is a town (e.g. Bolton, Dunkirk, Teesside)
  - A 1 in the matrix shows a **connection exists** between two towns
  - A 0 means no direct connection
- Example:
  - Bolton and Frogmore have a 1 at their intersection → they are connected
  - Dunkirk and Moulton have a 0 → no direct connection
- The matrix is **symmetric**, confirming that the graph is **undirected**

AI relevance

- Undirected, unweighted graphs are useful in AI for:
  - **Exploring environments**, e.g. mapping rooms in a building with equal-cost connections
  - **Clustering** in **machine learning**, where the presence of a connection implies similarity
  - **Social network analysis**, where mutual connections (friendships, links) are undirected
- These graphs are often used in:
  - **Depth-first** or **breadth-first search**
  - **Graph traversal algorithms** to explore all reachable nodes

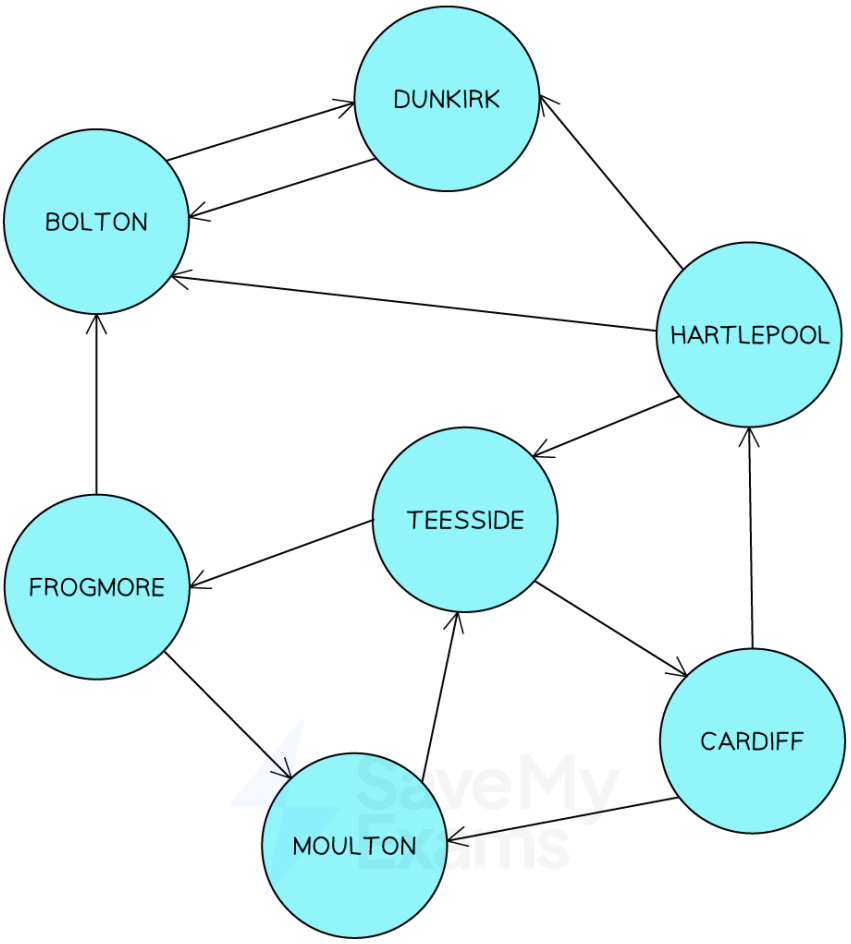
Directed, unweighted graph

- A directed graph (or digraph) is one where **edges have a direction**
- This means connections go from one node **to another**
- Connections **cannot be traversed in reverse** unless another directed edge exists

Your notes

© 2025 Save My Exams, Ltd.

Your notes



	B	C	D	F	H	M	T
B	0	0	1	0	0	0	0
C	0	0	0	0	1	1	0
D	1	0	0	0	0	0	0
F	1	0	0	0	0	1	0
H	1	0	1	0	0	0	1
M	0	0	0	0	0	0	1
T	0	1	0	1	0	0	0

Copyright © Save My Exams. All Rights Reserved

- In this example:
  - Each node represents a **town**

- Arrows show the **direction** of travel
- A 1 in the matrix at (row X, column Y) means there is a **directed edge from X to Y**
- A 0 means there is **no direct connection**

- For instance:
  - B → D is allowed (matrix value = 1)
  - D → C is not allowed (matrix value = 0)
- Because this matrix is **not symmetric**, it reflects the **one-way nature** of connections

AI relevance

- Directed, unweighted graphs are essential in AI for:

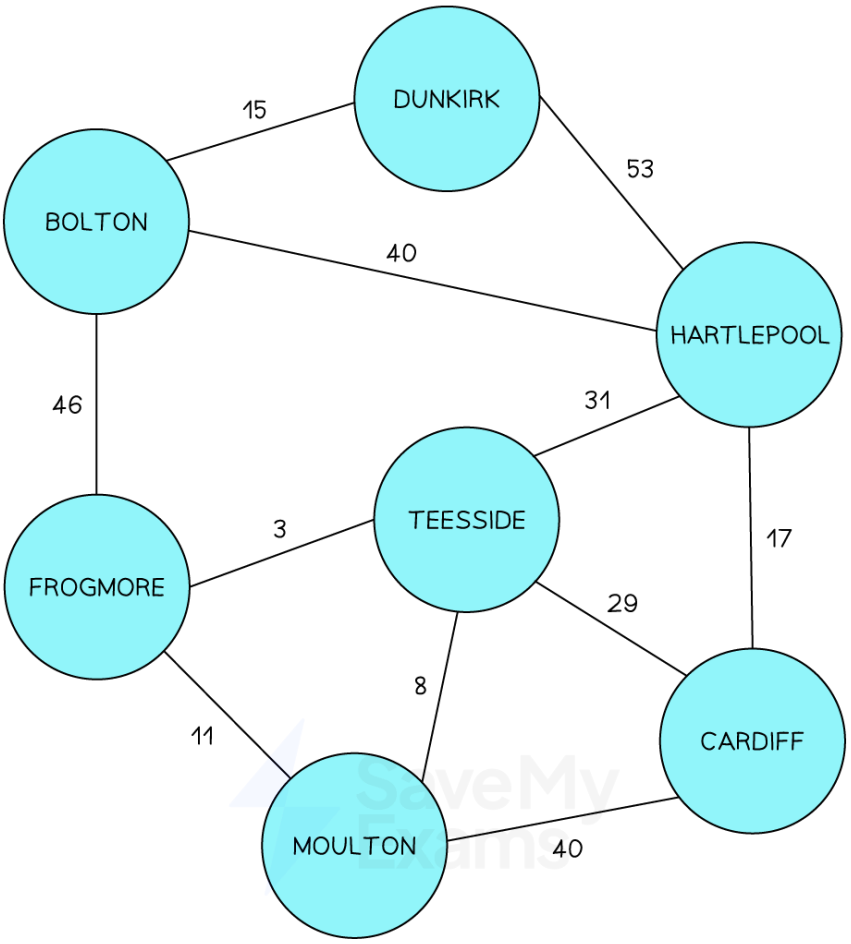
Use case	How the graph is used
Search algorithms	Used in **A***, **BFS**, **DFS** where state transitions are directional
Expert systems / rule engines	Representing dependencies between logical conditions or actions
Planning and scheduling	Modelling **prerequisites** and directed workflows
Navigation	Handling one-way streets or directed transport systems

Undirected, weighted graph

- An **undirected, weighted graph** is a graph where:
  - **Edges go both ways** (no direction)
  - **Each edge has a weight**, which can represent:
    - Cost
    - Distance
    - Time
    - Risk
- If no connection exists between two nodes, the matrix contains **∞ (infinity)**

Your notes

Your notes



	B	C	D	F	H	M	T
B	∞	∞	15	46	40	∞	∞
C	∞	∞	∞	∞	17	40	29
D	15	∞	∞	∞	53	∞	∞
F	46	∞	∞	∞	∞	11	3
H	40	17	53	∞	∞	∞	31
M	∞	40	∞	11	∞	8	8
T	∞	29	∞	3	31	∞	∞

Copyright © Save My Exams. All Rights Reserved

- Nodes represent **towns**
- Edges represent **roads**, and **weights** represent distances or costs

- In the matrix:
  - matrix[B][D] = 15 → Bolton to Dunkirk is 15 units
  - matrix[B][C] = ∞ → Bolton and Cardiff are not directly connected
- The matrix is **symmetric**, confirming this is an **undirected** graph

AI relevance

- Undirected, weighted graphs are widely used in AI when:

Use case	How It helps AI
Pathfinding (e.g. A*, Dijkstra)	Finding the **lowest cost path** between nodes
Clustering & similarity graphs	Edges represent **strength of relationships** or **similarity weights**
Recommendation engines	Connecting users or items based on weighted similarity
Robotics & navigation	Planning efficient movement paths through environments

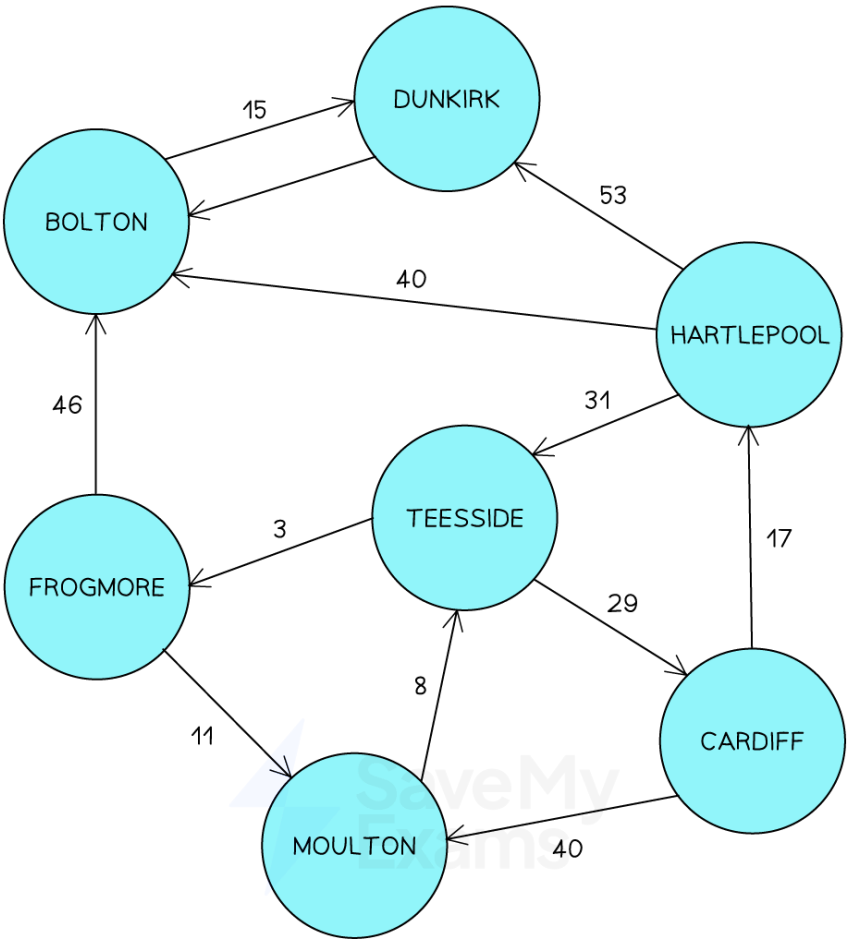
- In AI, the **weights** are crucial for deciding the **best route or option** when multiple paths are available

Directed, weighted graph

- A **directed, weighted graph** is a graph where:
  - **Edges have a direction** (e.g. A → B but not B → A)
  - Each edge has a **weight**, such as:
    - Distance
    - Cost
    - Time
    - Probability
- If there is **no edge**, the value is **∞ (infinity)**

Your notes

Your notes



	B	C	D	F	H	M	T
B	∞	∞	15	∞	∞	∞	∞
C	∞	∞	∞	∞	17	40	∞
D	15	∞	∞	∞	∞	∞	∞
F	46	∞	∞	∞	∞	11	∞
H	40	∞	53	∞	∞	∞	31
M	∞	∞	∞	∞	∞	∞	8
T	∞	29	∞	3	∞	∞	∞

Copyright © Save My Exams. All Rights Reserved

- Nodes = Towns
- Edges = One-way roads

- Weights = Travel cost or distance
- The adjacency matrix:
  - B → D = 15 → there is a road from Bolton to Dunkirk costing 15 units
  - C → H = 17 → Cardiff to Hartlepool has a weight of 17
  - H → T = 31 but T → H = ∞ → you can get from Hartlepool to Teesside, but not the other way
- This matrix is **not symmetric**, confirming the graph is **directed**

AI relevance

- Directed, weighted graphs are critical in AI for:

Use case	Purpose in AI
*A and Dijkstra algorithms**	Choosing the **shortest path** where direction and cost matter
Route planning in robotics	Modelling direction-sensitive travel (e.g. traffic rules)
Search algorithms	Tracking cost between **transitions between states**
Reinforcement learning	Modelling **rewards/costs** for state transitions in decision environments
Game AI	Movement systems with **terrain costs or restrictions**

Your notes

Your notes

Dijkstra's shortest path algorithm

- In Computer Science, an **optimisation problem** involves finding the **most efficient solution** to a given problem
- This could mean **minimising** cost, time, or resource usage, or **maximising** output, efficiency, or value
- Examples of optimisation problems include:
  - **Finding the shortest route** between two locations (e.g. Google Maps)
  - **Minimising resource usage** in manufacturing or video games
  - **Creating efficient timetables** (e.g. for schools or offices)
  - **Scheduling tasks** and staff shifts to avoid conflicts or downtime
- One of the most common real-world examples is **finding the shortest path from A to B**, which is exactly what Dijkstra's algorithm solves

What is Dijkstra's shortest path algorithm?

- In A Level Computer Science, **Dijkstra's shortest path algorithm** is a classic **optimisation algorithm**
- It calculates the shortest path from a **starting node** to **all other nodes** in a **weighted graph**
- It is often compared to **breadth-first search**, but Dijkstra's includes edge weights and ensures the **lowest total cost** to reach each destination node

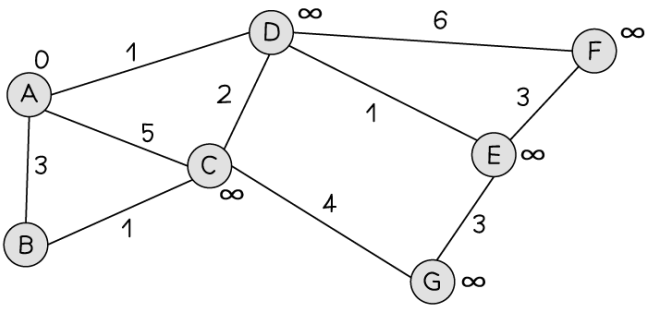
How It works:

- The graph is made up of **nodes (vertices)** and **edges (arcs)**
- Each edge has a **weight**, which can represent:
  - Time
  - Distance
  - Cost
- The algorithm **explores all possible routes**, keeping track of the shortest known distance to each node
- It guarantees the **optimal path** from the start node to every other node in the graph
- For revision on graphs, see section 18 - AI & Graphs
- An illustrated example of Dijkstra's algorithm is shown below:

Performing Dijkstra's shortest path algorithm

© 2025 Save My Exams, Ltd.

TARGET NODE: 6



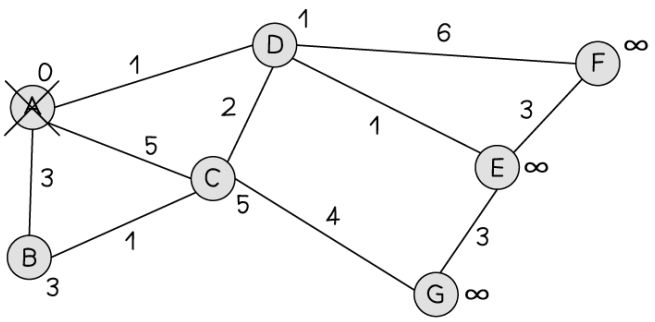
VISITED

Node	Weight + path	Previuos
A	0	–
B	∞	–
C	∞	–
D	∞	–
E	∞	–
F	∞	–
G	∞	–

Copyright © Save My Exams. All Rights Reserved

- Set A path to 0 and all other path weights to infinity

Your notes



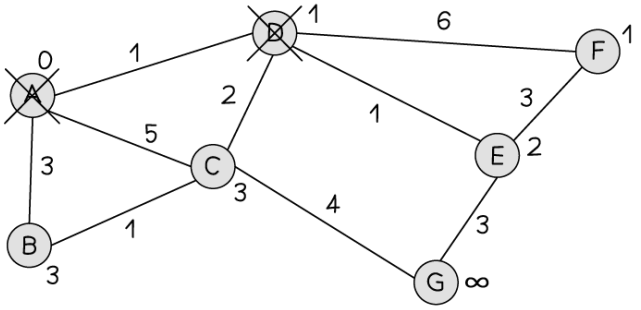
VISITED
A

Node	Weight + path	Previuos
~~A~~	~~0~~	~~–~~
B	3	–
C	5	–
D	1	–
E	∞	–
F	∞	–
G	∞	–

Copyright © Save My Exams. All Rights Reserved

- Visit A. Update all neighbouring nodes path weights to the distance from A + A's path weight (0)

Your notes



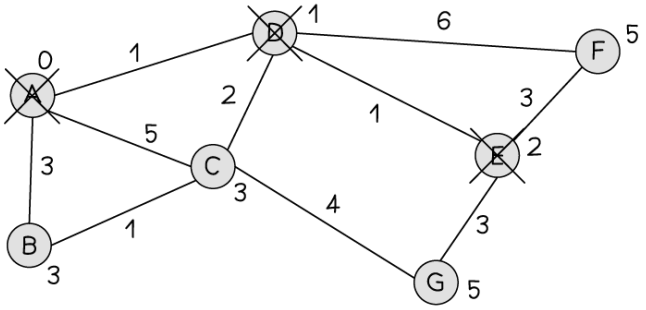
VISITED
AD

Node	Weight + path	Previuos
~~A~~	~~0~~	~~–~~
B	3	–
C	3	–
~~D~~	~~1~~	~~A~~
E	2	–
F	7	–
G	∞	–

Copyright © Save My Exams. All Rights Reserved

- Choose the next node to visit that has the lowest path weight (D)
- Visit D
- Update all neighbouring, non-visited nodes with a new path weight if the old path weight is bigger than the new path weight
- C is updated from 5 to 3 as the new path is shorted (A > D > C)

Your notes



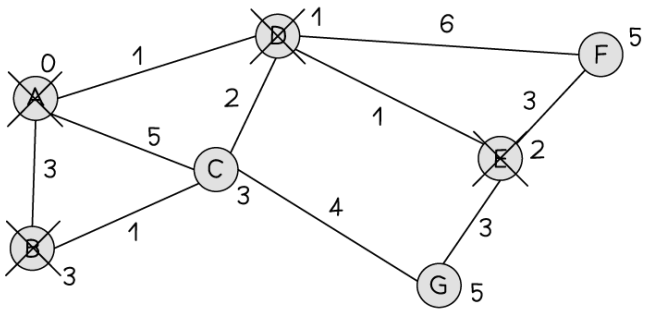
VISITED
ADE

Node	Weight + path	Previuos
~~A~~	~~0~~	~~–~~
B	3	–
C	3	–
~~D~~	~~1~~	~~A~~
~~E~~	~~2~~	~~D~~
F	5	–
G	5	–

Copyright © Save My Exams. All Rights Reserved

- Choose the next node to visit that has the lowest path weight (E)
- Visit E
- Update all neighbouring, non-visited nodes from E with a new path weight
- F is updated from 7 to 5 as the new path is shorter (A > D > E > F)

Your notes



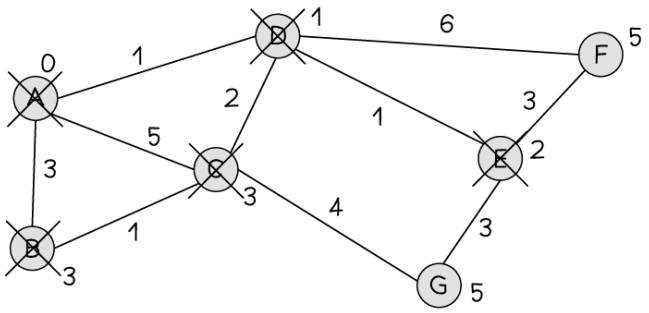
VISITED
ADEB

Node	Weight + path	Previuos
~~A~~	~~0~~	~~–~~
~~B~~	~~3~~	~~A~~
C	3	–
~~D~~	~~1~~	~~A~~
~~E~~	~~2~~	~~D~~
F	5	–
G	5	–

Copyright © Save My Exams. All Rights Reserved

- Choose the next node to visit, alphabetically (B)
- Visit B
- Update all neighbouring non-visited nodes from B with a new weight if the path is less than the current path
- C is not updated as A > B > C is 4

Your notes



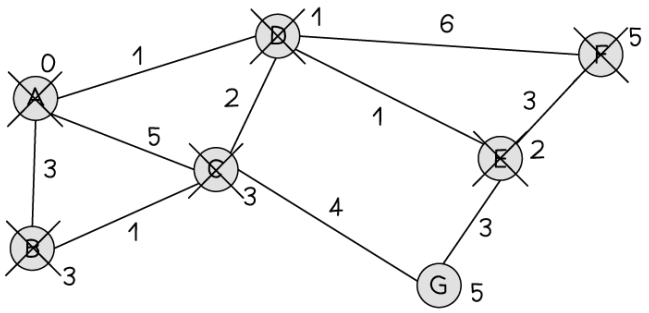
VISITED
ADEBC

Node	Weight + path	Previuos
~~A~~	~~0~~	~~–~~
~~B~~	~~3~~	~~A~~
~~C~~	~~3~~	~~D~~
~~D~~	~~1~~	~~A~~
~~E~~	~~2~~	~~D~~
F	5	–
G	5	–

Copyright © Save My Exams. All Rights Reserved

- Choose the next lowest node and visit (C)
- Update C's neighbours if the new path is shorter than the old path
- A > D > C > G is 8, so G is not updated

Your notes



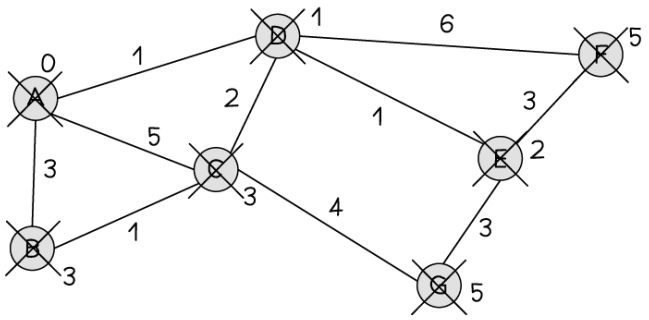
VISITED
ADEBCF

Node	Weight + path	Previuos
~~A~~	~~0~~	~~–~~
~~B~~	~~3~~	~~A~~
~~C~~	~~3~~	~~D~~
~~D~~	~~1~~	~~A~~
~~E~~	~~2~~	~~D~~
~~F~~	~~5~~	~~E~~
G	5	–

Copyright © Save My Exams. All Rights Reserved

- Choose the next lowest node and visit (F)
- Update F's neighbours if the new path is shorter than the old path
- F has no non-visited neighbours so no updates are made

Your notes



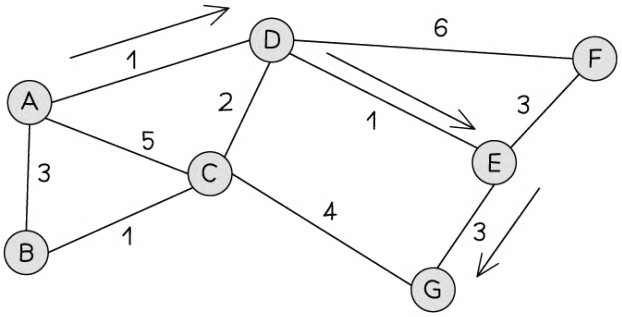
VISITED
ADEBCFG

Node	Weight + path	Previuos
~~A~~	~~0~~	~~–~~
~~B~~	~~3~~	~~A~~
~~C~~	~~3~~	~~D~~
~~D~~	~~1~~	~~A~~
~~E~~	~~2~~	~~D~~
~~F~~	~~5~~	~~E~~
~~G~~	~~5~~	~~E~~

Copyright © Save My Exams. All Rights Reserved

- Choose the next lowest node and visit (G)
- G has no non-visited neighbours so no updates are made
- All nodes have been visited
- Back tracking from G gives the following order and total distance A-D-E-G (5)

Your notes



VISITED
ADEBCFG

Node	Weight + path	Previous
A	0	–
B	3	A
C	3	D
D	1	A
E	2	D
F	5	E
G	5	E

Copyright © Save My Exams. All Rights Reserved

Figure 1: Performing Dijkstra's Shortest path Algorithm

Programming Dijkstra's shortest path algorithm

How do you write Dijkstra's shortest path algorithm?

Pseudocode

```
FUNCTION Dijkstra(graph, start, goal)

  // Initialise distances
  FOR EACH node FROM graph
    SET distance[node] TO infinity
  NEXT node

  SET distance[start] TO 0
  DECLARE previousNode AS DICTIONARY
  DECLARE visited AS LIST
  DECLARE unvisited AS LIST
```

Your notes

```
  FOR EACH node FROM graph
    ADD node TO unvisited
  NEXT node

  WHILE LENGTH(unvisited) > 0

    // Find unvisited node with shortest distance
    SET min TO null
    FOR EACH node FROM unvisited
      IF min = null THEN
        SET min TO node
      ELSEIF distance[node] < distance[min] THEN
        SET min TO node
      ENDIF
    NEXT node

    // Exit if goal reached
    IF min = goal THEN
      EXIT WHILE
    ENDIF

    // Update distances to neighbours
    FOR EACH neighbour FROM graph[min]
      SET cost TO weight(min, neighbour)  // assume this function returns the edge weight
      SET alt TO distance[min] + cost

      IF alt < distance[neighbour] THEN
        SET distance[neighbour] TO alt
        SET previousNode[neighbour] TO min
      ENDIF
    NEXT neighbour

    REMOVE min FROM unvisited
    ADD min TO visited

  ENDWHILE

  // Build path from goal to start
  DECLARE path AS LIST
  SET node TO goal

  WHILE node != start
    ADD node TO path
    SET node TO previousNode[node]
  ENDWHILE

  ADD start TO path
  REVERSE path

  RETURN path

ENDFUNCTION
```

Assumptions:

- graph[node] returns a list of neighbour nodes
- weight(x, y) returns the edge weight between node x and y

Your notes

- Lists like visited, unvisited, and path support ADD and REMOVE

Your notes

Python

```
def dijkstra(graph, start, goal):
    # Initialise distances and previous node map
    distance = {node: float('inf') for node in graph}
    distance[start] = 0
    previous_node = {}
    visited = set()
    unvisited = set(graph.keys())

    while unvisited:
        # Find the unvisited node with the smallest distance
        min_node = None
        for node in unvisited:
            if min_node is None or distance[node] < distance[min_node]:
                min_node = node

        if min_node == goal:
            break

        for neighbour, weight in graph[min_node]:
            if neighbour in visited:
                continue

            alt = distance[min_node] + weight
            if alt < distance[neighbour]:
                distance[neighbour] = alt
                previous_node[neighbour] = min_node

        visited.add(min_node)
        unvisited.remove(min_node)

    # Reconstruct path from goal to start
    path = []
    current = goal
    while current != start:
        path.append(current)
        current = previous_node[current]
    path.append(start)
    path.reverse()

    return path
```

Example usage:

```
# Graph format: adjacency list with weighted edges
graph = {
    'A': [('B', 2), ('C', 5)],
    'B': [('D', 1)],
    'C': [('D', 2)],
    'D': [('E', 3)],
    'E': []
}

print(dijkstra(graph, 'A', 'E'))
```

Output:

- A list of nodes representing the **shortest path** from 'A' to 'E'

Your notes

Your notes

A* algorithm

What is the A* algorithm?

- The **A*** (A-star) algorithm is a **pathfinding algorithm** that builds upon Dijkstra's algorithm
- It introduces a **heuristic function** to improve efficiency
- It is used to find the shortest path from a starting node to a goal node in a graph
- While Dijkstra's algorithm considers only the **actual cost** from the start node:
  - A* enhances this by estimating the **remaining distance** to the goal
  - This makes it **more goal-oriented** and often more efficient

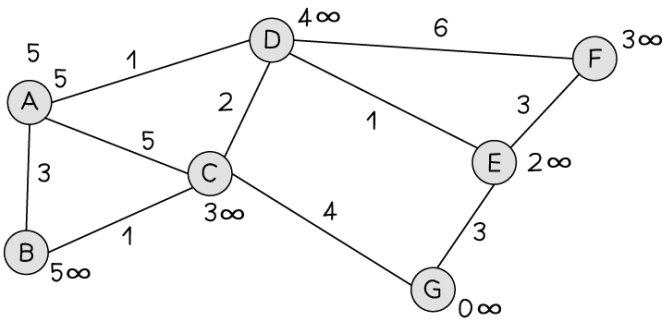
How A* improves on Dijkstra's

- To avoid inefficient detours, A* introduces a **heuristic function**, called **h(x)**
- It estimates the **straight-line distance** (Euclidean distance) from the current node to the goal
- A* combines both cost and estimate using the formula:
  - **f(x) = g(x) + h(x)**
    - **g(x)** = the actual cost from the start node to the current node
    - **h(x)** = the heuristic estimate to the goal node
    - **f(x)** = the total estimated cost of the cheapest solution through node x
- By choosing nodes with the lowest **f(x)** value, A* aims to explore paths that are both cheap and **move closer to the goal**

Heuristics in A*

- The heuristic function **h(x)** should **never overestimate** the true cost to the goal
- This ensures A* remains **optimally efficient**
- The closer h(x) is to the true cost, the **fewer nodes** A* needs to explore
- If h(x) increases, it may indicate the algorithm is moving **away from the goal**, helping it backtrack or choose a better path
- Below is an illustration of the A* search:

© 2025 Save My Exams, Ltd.

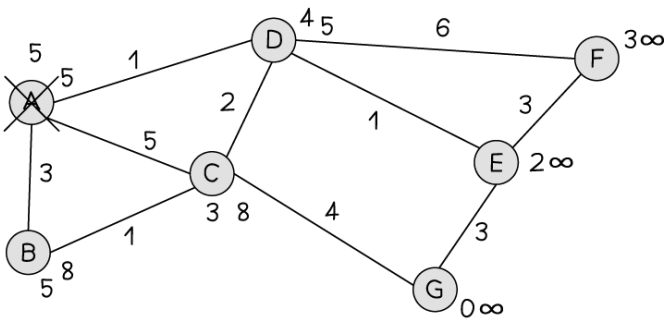


VISITED

Node	g(x)	h(x)	f(x)	Previuos
A	0	5	5	–
B	∞	5	∞	–
C	∞	3	∞	–
D	∞	4	∞	–
E	∞	2	∞	–
F	∞	3	∞	–
G	∞	0	∞	–

Copyright © Save My Exams. All Rights Reserved

Your notes

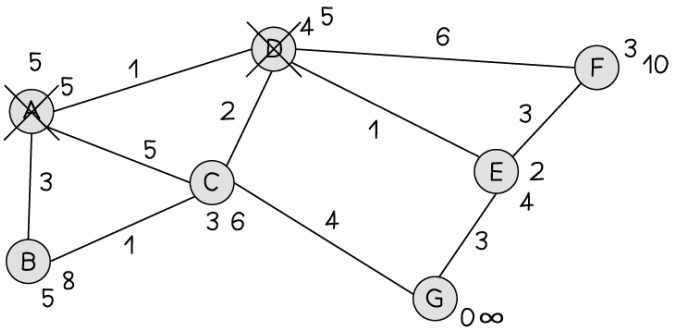


VISITED
A

Node	g(x)	h(x)	f(x)	Previous
~~A~~	~~0~~	~~5~~	~~5~~	~~–~~
B	3	5	8	–
C	5	3	8	–
D	1	4	5	–
E	∞	2	∞	–
F	∞	3	∞	–
G	∞	0	∞	–

Copyright © Save My Exams. All Rights Reserved

Your notes

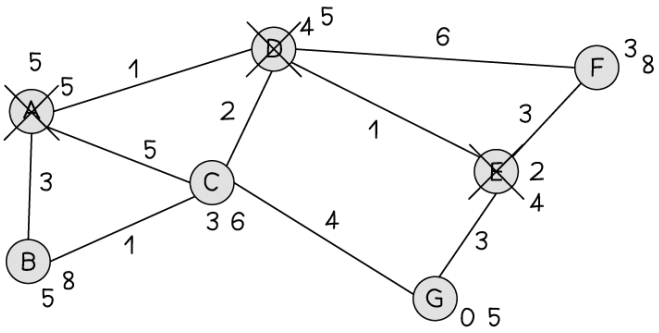


VISITED
AD

Node	g(x)	h(x)	f(x)	Previous
~~A~~	~~0~~	~~5~~	~~5~~	~~–~~
B	3	5	8	–
C	3	3	6	–
~~D~~	~~1~~	~~4~~	~~5~~	~~A~~
E	2	2	4	–
F	7	3	10	–
G	∞	0	∞	–

Copyright © Save My Exams. All Rights Reserved

Your notes

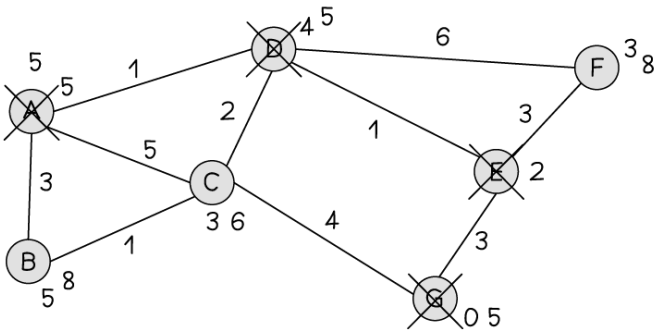


VISITED
ADE

Node	g(x)	h(x)	f(x)	Previous
~~A~~	~~0~~	~~5~~	~~5~~	~~–~~
B	3	5	8	–
C	3	3	6	–
~~D~~	~~1~~	~~4~~	~~5~~	~~A~~
~~E~~	~~2~~	~~2~~	~~4~~	~~D~~
F	5	3	8	–
G	5	0	5	–

Copyright © Save My Exams. All Rights Reserved

Your notes

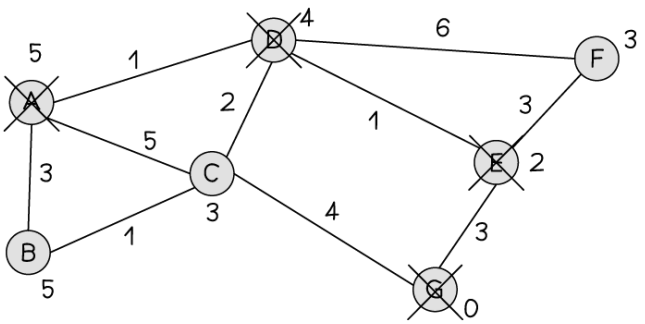


VISITED
ADEG

Node	g(x)	h(x)	f(x)	Previous
~~A~~	~~0~~	~~5~~	~~5~~	~~–~~
B	3	5	8	–
C	3	3	6	–
~~D~~	~~1~~	~~4~~	~~5~~	~~A~~
~~E~~	~~2~~	~~2~~	~~4~~	~~D~~
F	5	3	8	–
~~G~~	~~5~~	~~0~~	~~5~~	~~E~~

Copyright © Save My Exams. All Rights Reserved

Your notes



VISITED
ADEG

Node	g(x)	h(x)	f(x)	Previous
A	0	5	5	–
B	3	5	8	–
C	3	3	6	–
D	1	4	5	A
E	2	2	4	D
F	5	3	8	–
G	5	0	5	E

Copyright © Save My Exams. All Rights Reserved

Figure 2: Performing the A* Search

A* vs Dijkstra's

Feature	Dijkstra's	A* Search
Goal-aware?	No	Yes
Cost function	g(x)	g(x) + h(x)
Speed	Slower (explores more)	Faster (more direct toward goal)
Use of heuristic	None	Yes, estimates distance to goal

Your notes

Best for...	Full shortest path map	Fastest route to a specific destination

Your notes

Programming A* algorithm

How do you write an A* algorithm?

Pseudocode

```
FUNCTION AStarSearch(graph, start, goal)

  // Initialise distances and f-scores
  FOR EACH node FROM graph
    SET g[node] TO infinity
    SET f[node] TO infinity
  NEXT node

  SET g[start] TO 0
  SET f[start] TO h[start]

  DECLARE openSet AS LIST
  ADD start TO openSet

  WHILE openSet IS NOT EMPTY

    // Find node in openSet with the lowest f value
    SET min TO null
    FOR EACH node FROM openSet
      IF min = null THEN
        SET min TO node
      ELSEIF f[node] < f[min] THEN
        SET min TO node
      ENDIF
    NEXT node

    IF min = goal THEN
      EXIT WHILE
    ENDIF

    REMOVE min FROM openSet

    // Check all neighbours of current node
    FOR EACH neighbour FROM graph[min]
      SET tentativeG TO g[min] + cost(min, neighbour)

      IF tentativeG < g[neighbour] THEN
        SET g[neighbour] TO tentativeG
        SET f[neighbour] TO g[neighbour] + h[neighbour]
        SET previousNode[neighbour] TO min

        IF neighbour NOT IN openSet THEN
          ADD neighbour TO
```

- graph[min] assumes you're storing adjacency lists
- h[node] is your heuristic table
- g[] and f[] are maps (dictionaries) for real cost and estimated total cost

- cost(min, neighbour) should be defined or assumed available

Python

```
def a_star_search(graph, start, goal, h):
    open_set = [start]
    g = {node: float('inf') for node in graph}
    f = {node: float('inf') for node in graph}
    came_from = {}

    g[start] = 0
    f[start] = h[start]

    while open_set:
        # Get node with lowest f value
        current = min(open_set, key=lambda node: f[node])

        if current == goal:
            break

        open_set.remove(current)

        for neighbour, cost in graph[current]:
            tentative_g = g[current] + cost

            if tentative_g < g[neighbour]:
                g[neighbour] = tentative_g
                f[neighbour] = g[neighbour] + h[neighbour]
                came_from[neighbour] = current

                if neighbour not in open_set:
                    open_set.append(neighbour)

    # Reconstruct path
    path = []
    node = goal
    while node != start:
        path.append(node)
        node = came_from[node]
    path.append(start)
    path.reverse()

    return path
```

Example usage:

```
graph = {
    'A': [('B', 2), ('C', 5)],
    'B': [('D', 4)],
    'C': [('D', 1)],
    'D': [('E', 3)],
    'E': []
}

h = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 2,
    'E': 0
```

Your notes

```
}

print(a_star_search(graph, 'A', 'E', h))
```

Output:

- A possible shortest path from 'A' to 'E', guided by the heuristic values

Your notes

Your notes

Neural networks & deep learning

What is machine learning (ML)?

- Machine learning is **a type of artificial intelligence (AI)** that allows computers to:
  - **Learn patterns from data**
  - Improve performance **without being explicitly programmed**
- Instead of following fixed rules, machine learning systems:
  - **Analyse** large amounts of data
  - **Identify** patterns or trends
  - **Make** predictions or decisions based on that data
- ML is used in: spam filters, voice assistants, recommendation systems, fraud detection

What are artificial neural networks (ANNs)?

- Artificial neural networks (ANNs) are **algorithms** inspired by the structure of the **human brain**
  - A neural network is made up of **layers of nodes (neurons)** connected by **weighted links**
  - Each neuron receives input, processes it, and passes the result to the next layer
  - The network **adjusts the weights** based on errors in output (using algorithms like backpropagation)

How ANNs help machine learning

- ANNs are powerful because they:
  - Automatically **learn from experience**, even with complex or unstructured data
  - Improve their accuracy with **more data and training**
  - Can solve problems that are **too complex for rule-based programming**
- Examples: handwriting recognition, speech recognition, image classification, language translation

Deep learning (DL)

- Deep learning is a **subfield of machine learning** that uses **deep (multi-layered) neural networks**
  - The more layers a network has, the more complex patterns it can learn
  - Especially effective at learning **abstract features** in images, text, audio, and video

© 2025 Save My Exams, Ltd.

- Example: facial recognition apps learn features like eyes, noses, and then full faces through many layers

Your notes

Reinforcement learning (RL)

- Reinforcement learning is another type of machine learning where an **agent learns by interacting with an environment**
  - It **receives rewards** for good actions and **penalties** for poor actions
  - Over time, the agent learns an **optimal strategy** (called a *policy*) to maximise rewards
- Used in: robotics, self-driving cars, game-playing AI (like AlphaGo), industrial automation

Summary: why use these methods?

Method	What it does	Why use it
Machine Learning	Learns from labelled data	Automates decisions with high accuracy
Artificial Neural Network	Mimics human brain to recognise complex patterns	Excellent for handling messy, high-dimensional data
Deep Learning	Uses many neural layers to extract features	Performs well with images, language, sound
Reinforcement Learning	Learns by trial and error with feedback	Useful where there's no labelled data, only success/failure

Categories of machine learning

- Machine learning algorithms are categorised based on **how they learn from data**
- The two main categories are:

Supervised learning

- Supervised learning is when the algorithm is trained on a **labelled dataset**
- Meaning the **input data** has **known outputs** (answers)
  - The goal is to learn a function that maps inputs to the correct output
  - Once trained, the model can make predictions on **unseen data**
- Example:
  - Input: Email text
  - Output: Spam or Not Spam

© 2025 Save My Exams, Ltd.

- The model learns from thousands of pre-labelled emails

Used for:

- **Classification** (e.g. face recognition, spam detection)
- **Regression** (e.g. predicting house prices)

Unsupervised learning

- Unsupervised learning is when the algorithm is given **unlabelled data**, and it must **find patterns** or **groupings** on its own
  - There are no correct answers, the system explores the data structure
  - It identifies **clusters, trends, or anomalies** without any guidance
- Example:
  - Input: Customer purchase data
  - Output: Groups of customers with similar buying habits

Used for:

- **Clustering** (e.g. customer segmentation, social network analysis)
- **Dimensionality reduction** (e.g. simplifying complex data)

Back propagation & regression

What is back propagation?

- Back propagation is a training method used in **artificial neural networks** to **improve accuracy** by adjusting the weights of connections
- It is a key part of the **supervised learning** process and works as follows:

1. **Forward pass**:
   1. Input data passes through the network layer by layer
   2. The network produces an **output** (a prediction)
2. **Error calculation**
   1. The output is compared to the **actual target** value
   2. The **difference** is called the **error**
3. **Backward pass**
   1. The error is **propagated backwards** through the network
   2. Each layer calculates its **contribution to the error**
4. **Weight adjustment**
   1. The weights of the connections are updated using an algorithm (e.g. **gradient descent**) to reduce future errors

Your notes

Why use back propagation?

- Helps the neural network **learn from mistakes**
- Makes the model more **accurate over time**
- Allows multi-layer networks to **fine-tune all layers**, not just the output
- Common in: image recognition, speech recognition, language translation

What is regression?

- Regression is a type of **supervised learning** used to predict **continuous values** (rather than categories)
- The aim is to **find the relationship** between input features and a numerical output

Types of regression:

Type	Description
Linear regression	Predicts output using a straight-line relationship ($y = mx + c$)
Multiple linear regression	Uses **multiple input features** to predict one continuous output
Logistic regression	Predicts **binary outcomes** (e.g. yes/no), despite the name "regression"

Example: linear regression

- Predicting house prices based on square footage:
  - Input: size of house
  - Output: estimated price
  - The model learns a line that best fits the data points

Why use regression methods?

- Useful for **forecasting**, **trend analysis**, and **risk prediction**
- Interpretable, often used in real-world problems like:
  - Business analytics
  - Finance
  - Healthcare

Worked Example

Your notes

© 2025 Save My Exams, Ltd.

Supervised and unsupervised learning are two categories of machine learning.

Describe supervised learning and unsupervised learning. **[4]**

Answer

Supervised learning (Max 3 of 4)

- Supervised learning allows data to be collected, or a data output produced, from the previous experience [1 mark]
- In supervised learning, known input and associated outputs are given **OR** uses sample data with known outputs (in training) **OR** uses labelled input data [1 mark]
- Able to predict future outcomes based on past data [1 mark]

Unsupervised learning (Max 3 of 4)

- Unsupervised machine learning helps all kinds of unknown patterns in data to be found [1 mark]
- Unsupervised learning only requires input data to be given [1 mark]
- Uses any data **OR** not trained on the right output **OR** uses unlabelled input data [1 mark]

Your notes